

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

Отчёт
по лабораторной работе №1
по курсу “Логика и основы алгоритмизации в инженерных задачах”
на тему “простые структуры данных”

Выполнил:
Студент группы 25ВВВЗ
Мартьянов А.Ю.
Дмитриев С.А.

Приняли:
Юрова О.В.
Деев М.В.

Пенза 2026

Название

Простые структуры данных

Цель работы

Вспомнить материал за прошлый курс и разработать программу, работая с массивами и структурами

Лабораторное задание

1. написать программу, вычисляющую разницу между максимальным и минимальным элементами массива
2. написать программу, реализующую инициализацию массива случайными числами
3. написать программу, реализующую создание массива произвольного размера, вводимого с клавиатуры
4. написать программу, вычисляющую сумму значений в каждом столбце (или строке) двумерного массива
5. написать программу, осуществляющую поиск среди структур student структуру с заданными параметрами (фамилией, именем и т.д.).

Ход работы

1. Вычисление разницы между максимальным и минимальным элементами массива

Использованные функции и операторы: malloc(), free(), scanf(), printf(), цикл for, условный оператор if.

Описание работы: С помощью функции scanf() считывается размер массива. Функция malloc() динамически выделяет блок памяти в куче для хранения элементов. В цикле for происходит поэлементный ввод данных. С помощью условного оператора if внутри цикла сравниваются элементы: если текущий элемент меньше текущего минимума, он становится новым минимумом (аналогично для максимума). В конце вычисляется разность, а функция free() освобождает выделенную память.

```

=== МЕНЮ ПРОГРАММЫ ===
1. Разница max и min
2. Массив, случайные числа
3. Динамический массив
4. Сумма значений строки/столбцы
5. Ввод данных студентов
0. Выход
Выберите пункт меню: 1

=== Разница между max и min элементами массива ===
Введите размер массива (1-100): 5
Введите 5 элементов массива:
Элемент 1: 53
Элемент 2: 12
Элемент 3: 74
Элемент 4: 433
Элемент 5: 85
Массив: [53, 12, 74, 433, 85]
Минимальный элемент: 12
Максимальный элемент: 433
Разница: 433 - 12 = 421

```

Рисунок 1

2. Инициализация массива случайными числами

Использованные функции и операторы: `srand()`, `time()`, `rand()`, оператор `%` (остаток от деления), `malloc()`, `free()`.

Описание работы: Функция `srand()` в связке с `time(NULL)` инициализирует генератор псевдослучайных чисел текущим системным временем, что гарантирует новую последовательность при каждом запуске. Функция `rand()` генерирует случайное число. Оператор `%` (взятие остатка от деления) используется для ограничения диапазона генерации, а сложение с минимальным значением сдвигает этот диапазон. Выделение и очистка памяти осуществляются через `malloc()` и `free()`.

```

=== МЕНЮ ПРОГРАММЫ ===
1. Разница max и min
2. Массив, случайные числа
3. Динамический массив
4. Сумма значений строки/столбцы
5. Ввод данных студентов
0. Выход
Выберите пункт меню: 2

=== Инициализация массива случайными числами ===
Введите размер массива (1-100): 5
Введите минимальное значение: 1
Введите максимальное значение: 987
Сгенерированный массив: [664, 593, 30, 488, 262]

```

Рисунок 2

3. Создание массива произвольного размера, вводимого с клавиатуры

Использованные функции и операторы: `malloc()`, `sizeof()`, `free()`, `scanf()`.

Описание работы: Размер массива запрашивается у пользователя через `scanf()`. Функция `sizeof(int)` возвращает размер одного элемента типа `int` в байтах. Команда `malloc(size * sizeof(int))` вычисляет общий требуемый объем памяти и запрашивает его у операционной системы во время выполнения программы. Это позволяет создать массив, размер которого не был известен на этапе компиляции. После завершения работы с массивом команда `free()` возвращает память системе.

```

=== МЕНЮ ПРОГРАММЫ ===
1. Разница max и min
2. Массив, случайные числа
3. Динамический массив
4. Сумма значений строки/столбцы
5. Ввод данных студентов
9. Выход
Выберите пункт меню: 3

=== Создание массива произвольного размера ===
Введите размер массива (1-100): 5
Введите 5 элементов массива:
Элемент 1: 75
Элемент 2: 18
Элемент 3: 98
Элемент 4: 23
Элемент 5: 1
Созданный массив: [75, 18, 98, 23, 1]

```

Рисунок 3

4. Вычисление суммы значений в каждом столбце и строке двумерного массива

Использованные функции и операторы: Статический двумерный массив, вложенные циклы for, оператор += (сложение с присваиванием).

Описание работы: Матрица объявляется как статический двумерный массив фиксированного размера. Для вычисления сумм применяются вложенные циклы for. При подсчете суммы строк внешний цикл итерирует по индексам строк, а внутренний — по столбцам, накапливая сумму через оператор +=. Для подсчета суммы столбцов порядок циклов меняется местами (внешний цикл идет по столбцам, внутренний — по строкам), что позволяет последовательно проходить по каждому вертикальному срезу матрицы.

```

=== МЕНЮ ПРОГРАММЫ ===
1. Разница max и min
2. Массив, случайные числа
3. Динамический массив
4. Сумма значений строки/столбцы
5. Ввод данных студентов
9. Выход
Выберите пункт меню: 4

=== Сумма значений в строках и столбцах матрицы ===
Введите количество строк (1-10): 5
Введите количество столбцов (1-10): 5
Введите минимальное значение для случайных чисел: 1
Введите максимальное значение для случайных чисел: 88

Сгенерированная матрица 5x5:
76 64 39 54 79
58 22 14 69 20
67 87 54 13 65
12 8 14 3 78
31 72 86 56 68

Суммы строк:
Строка 1: 312
Строка 2: 183
Строка 3: 286
Строка 4: 115
Строка 5: 313

Суммы столбцов:
Столбец 1: 244
Столбец 2: 253
Столбец 3: 207
Столбец 4: 195
Столбец 5: 310

```

Рисунок 4

5. Осуществление поиска среди структур student по заданным параметрам

Использованные функции и операторы: typedef struct, массив структур, scanf(), библиотечная функция strcmp(), оператор ==, переменная-флаг (found).

Описание работы: С помощью ключевого слова typedef struct создается пользовательский тип данных Student, объединяющий строки и числа. Создается массив таких структур. Для поиска по строковым полям (фамилия, имя) применяется функция strcmp(), которая сравнивает строки посимвольно и возвращает 0, если они полностью идентичны. Для числовых полей (возраст, балл) используется стандартный оператор сравнения ==. Переменная-флаг found инициализируется нулем и меняется на 1, если хотя бы один студент найден, что позволяет вывести сообщение об отсутствии результатов.

```
=== МЕНЮ ПРОГРАММЫ ===
1. Разница max и min
2. Массив, случайные числа
3. Динамичный массив
4. Сумма значений строки/столбцы
5. Ввод данных студентов
0. Выход
Выберите пункт меню: 5

=== Ввод данных студентов и поиск ===
Введите количество студентов (1-100): 3
Введите данные 3 студентов (используя английские символы):

Студент 1:
Фамилия: kiske
Имя: ku
Возраст: 65
Средний балл: 32

Студент 2:
Фамилия: hoffen
Имя: harry
Возраст: 23
Средний балл: 1

Студент 3:
Фамилия: lieben
Имя: larry
Возраст: 67
Средний балл: 5

Введенные данные студентов:
1. kiske ku, возраст: 65, средний балл: 32,0
2. hoffen harry, возраст: 23, средний балл: 1,0
3. lieben larry, возраст: 67, средний балл: 5,0

=== ПОИСК СТУДЕНТОВ ===
1 - Поиск по фамилии
2 - Поиск по имени
3 - Поиск по возрасту
4 - Поиск по среднему баллу
0 - Вернуться в главное меню
Выберите тип поиска: 1
Введите фамилию для поиска: hoffen

Результаты поиска по фамилии 'hoffen':
Найден: hoffen harry, возраст: 23, средний балл: 1,0
```

Рисунок 5

Вывод

В ходе выполнения лабораторной работы были разработаны программы, выполняющие указанные в лабораторной работе задачи. Результаты работ программ совпали с результатами трассировок, следовательно, программы работают без ошибок. Мы получили опыт в создании проектов в среде Microsoft Visual Studio.